

Heterogeneous Parallel Programming: Dive into the World of CUDA

A previous article in this series titled 'Introducing NVIDIA's CUDA' covered the basics of the NVIDIA CUDA device architecture. This article covers parallel programming using CUDA C with sequential and parallel implementations of a vector addition program.



Parallel programming and general-purpose GPU computing are some of the hottest trends in computer science today due to the decreased prices of multi-core systems and the increase in compute efficiency. Various parallel programming languages like OpenCL and CUDA have been developed and evaluated over the years. This article will cover the basics of CUDA C, invoking kernels, threads and blocks with a vector addition program and it aims to give you an insight into beginning programming on your CUDA device.

A few important points before we begin. To use CUDA on your system, you will need to have the following installed:

1. CUDA-capable GPU hardware
2. A supported version of Linux with a GCC compiler and toolchain
3. NVIDIA CUDA toolkit and drivers

It is presumed that if you already have a CUDA device within your system, you probably have the latest toolkit and drivers installed and configured correctly. In case you do not have the NVIDIA CUDA drivers configured or you have recently upgraded your hardware with a CUDA device, you could follow the simple steps given below to configure your device.

1. Download the toolkit from the NVIDIA website, available

at no cost from: <http://www.nvidia.com/content/cuda-downloads.html>. Select the correct product release depending on your operating system preferences. This download contains an all-in-one package, which includes the CUDA toolkit, SDK code samples and the required drivers. After downloading, follow the steps in the NVIDIA guide to install the drivers, CUDA samples and the toolkit, at: http://developer.download.nvidia.com/compute/cuda/5_0/rel/docs/CUDA_Getting_Started_Guide_For_Linux.pdf. This guide will help you set up the complete environment on the system.

In this article, I will cover the serial and parallel versions of a vector addition program. Once you have understood the basics of the parallel vector addition program, you could use the concepts pretty well in parallelising other algorithms as per your requirements.

First, let's write a simple C program to perform vector addition of two arrays. Open your favourite editor and write a simple vector addition code that looks like what's shown below:

```
#include<stdio.h>

static const int N=100;
```